



OOP DESIGN, CORE JS LOGIC (FUNCTIONS, ARRAYS/LOOPS/CONDITI ONALS)

STUDENT SUPPORT PORTAL

Pseudocode

T4 : Object Oriented Design Deliverable : CLASS DIAGRAM

```
1
2  /// T4: OBJECT-ORIENTED DESIGN - 2+ CLASSES [8 Marks]
3  /// Deliverable: Class Diagram*
4  +-----+ 1 *+-----+
5  | User |<-----| Subject |
6  +-----+ +-----+
7  | - userID | | - subjectID |
8  | - name | | - subjectName |
9  | - email | | - material[] |
10 | - password | +-----+
11 | - profilePhoto |
12 +-----+
13 | ^
14 | |
15 +---+---+
16 | |
17 +-----+ 1 *+-----+ 1 * +-----+
18 | Learner |-----| Assignment |<-----| Submission |
19 +-----+ +-----+ +-----+
20 | - classID | | - assignID | | - submissionID |
21 +-----+ | - title | | - grade |
22 | | | | | | | - dueDate | | - feedback |
23 | | | | | | | - status | | - submitDate |
24 | | | | | | +-----+ +-----+
25 | ^
26 | |
27 +-----+
28 | Lecturer |
29 +-----+
30 | - staffID |
31 | - classes[] |
32 +-----+
33 | +markRegister() |
34 | +grade() |
35 | +uploadMaterial |
36 +-----+
37
38 Relationships:
39 1. User` is parent of `Learner` and `Lecturer` - Inheritance
40 2. Lecturer 1---_ Assignment` - Lecturer creates many assignments
41 3. Learner 1---_ Submission` - Learner submits many assignments
42 4. Assignment 1---* Submission` - One assignment has many submissions
```

Pseudocode

2

T8: VARIABLES, DATA TYPES & OPERATORS

```
45  T8: VARIABLES, DATA TYPES & OPERATORS [8 Marks]
46
47  Deliverable: Real code snippet - Python example
48  Variables and Data Types
49  learner_name = "Insert learner name" # string
50  learner_id = 2025001 # int
51  average_mark = 78.5 # float
52  is_active = True # boolean
53  subjects = ["Insert subjects"] # list/array
54
55  Operators
56  total_assignments = 10
57  completed = 7
58  completion_rate = (completed / total_assignments) * 100 # arithmetic
59
60  Type Conversion
61  marks_str = "85"
62  marks_int = int(marks_str) # string to int
63  attendance_text = str(92) + # int to string
64
65  Conditional Operator
66  status = "PASS" if average_mark >= 50 else "FAIL"
```

Pseudocode

3

T9: FUNCTIONS AND SCOPE

Deliverable: PSEUDOCODE first for each function

```
68  T9: FUNCTIONS AND SCOPE [10 Marks]
69  ///< Deliverable: PSEUDOCODE first for each function
70  // GLOBAL VARIABLE
71  currentUser = null
72
73  ▾ FUNCTION login(email, password)
74      // LOCAL VARIABLES: email, password
75      user = database.findUser(email)
76  ▾  IF user exists AND user.password == password THEN
77      SET currentUser = user
78  ▾      IF user.role == "Learner" THEN
79          RETURN redirectTo("Learner_Overview")
80  ▾      ELSE IF user.role == "Lecturer" THEN
81          RETURN redirectTo("Lecturer_Overview")
82      END IF
83  ▾  ELSE
84      incrementFailedAttempts()
85  ▾      IF failedAttempts == 3 THEN
86          SHOW "Forgot password?" IN RED
87      END IF
88      RETURN "Error: Incorrect credentials"
89  END IF
90  END FUNCTION
91
92  ▾ FUNCTION resetPassword(email)
93      // LOCAL VARIABLES: email, otp
94  ▾  IF email is registered THEN
95      SEND otp TO email
96      WAIT FOR newPassword
97      UPDATE user.password = newPassword
98      INVALIDATE old password
99      RETURN "Password updated. Go to Login"
100  END IF
101  END FUNCTION
102
```

```
102
103  FUNCTION updateProfile(userID, newData)
104      // LOCAL VARIABLES: userID, newData
105      REQUIRE re-authentication IF newData.email is changed
106      UPDATE database.user WHERE id = userID WITH newData
107      REFRESH app.header WITH newData
108      RETURN "Profile updated"
109  END FUNCTION
110
111
```

Pseudocode



4

Pseudocode

5 T10: ARRAYS, LOOPS & CONDITIONALS

Deliverable: Pseudocode the logic before coding

I.ASSIGNMENT STATUS

```
112 T10: ARRAYS, LOOPS & CONDITIONALS [12 Marks]
113 Deliverable: Pseudocode the logic before coding
114 // 1. ASSIGNMENT STATUS COLOR CODING
115 ∨ FUNCTION updateAssignmentColors(assignmentList[])
116     // assignmentList is an ARRAY
117     currentDate = getToday()
118
119 ∨     FOR i = 0 TO length(assignmentList) - 1 DO
120         assignment = assignmentList[i]
121
122 ∨         IF assignment.isSubmitted == false AND assignment.dueDate < currentDate THEN
123             assignment.statusColor = "RED" // Overdue
124 ∨         ELSE IF assignment.isSubmitted == true AND assignment.submitDate <= assignment.dueDate THEN
125             assignment.statusColor = "GREEN" // On time
126 ∨         ELSE IF assignment.isSubmitted == false AND assignment.dueDate >= currentDate THEN
127             assignment.statusColor = "GREY" // Not due yet
128         END IF
129     END FOR
130
131     RETURN assignmentList
132 END FUNCTION
133
134
```

Pseudocode

6

2. CLASS REGISTER + ATTENDANCE GRAPH UPDATE

```
134
135 // 2. CLASS REGISTER + ATTENDANCE GRAPH UPDATE
136 FUNCTION markAttendance(classID, date, attendanceRecords[])
137     // attendanceRecords is ARRAY of {learnerID, status}
138     FOR EACH record IN attendanceRecords DO
139         SAVE record TO database
140         UPDATE learner.attendanceHistory ADD record
141     END FOR
142
143     // Trigger graph update for all learners in class
144     CALL updateLearnerAttendanceGraph(classID)
145 END FUNCTION
146
147
```

3. PROGRESS STATEMENT - FLAG UNDERPERFORMERS

```
147
148 // 3. PROGRESS STATEMENT - FLAG UNDERPERFORMERS
149 ∨ FUNCTION generateProgressStatement(learnerID)
150     attendanceRate = calculateAttendance(learnerID)
151     assignmentAverage = calculateAssignmentAverage(learnerID)
152
153 ∨ IF attendanceRate < 70 OR assignmentAverage < 50 THEN
154     flag = "NEEDS EXTRA HELP"
155     area = identifyWeakArea(learnerID)
156 ∨ ELSE
157     flag = "ON TRACK"
158     area = null
159 END IF
160
161     RETURN {attendanceRate, assignmentAverage, flag, area}
162 END FUNCTION
163
```

Pseudocode

7

Review , Reflection & Explanation

- **Book assessor session, log feedback, write contribution summary Book assessor session, log feedback, write contribution summary**

Pseudocode

8

ANNOTATED WIREFRAMES

[P12-1 COOKIE]: On "Sign In to Dashboard", save preference cookie:

```
document.cookie = "userEmail=alex.learner@skillstrack.in;  
expires=7days"
```

Purpose: Remember user email for next login.

[P12-3 REDIRECT]: On successful login, redirect to Learner Dashboard:

```
window.location.href = "learner-dashboard.html"
```

[P12-3 REDIRECT]: On successful login, redirect to Assessor Overview:

```
window.location.href = "assessor-overview.html"
```

[P12-1 COOKIE]: On "Create Account", set cookie to track first-time user:

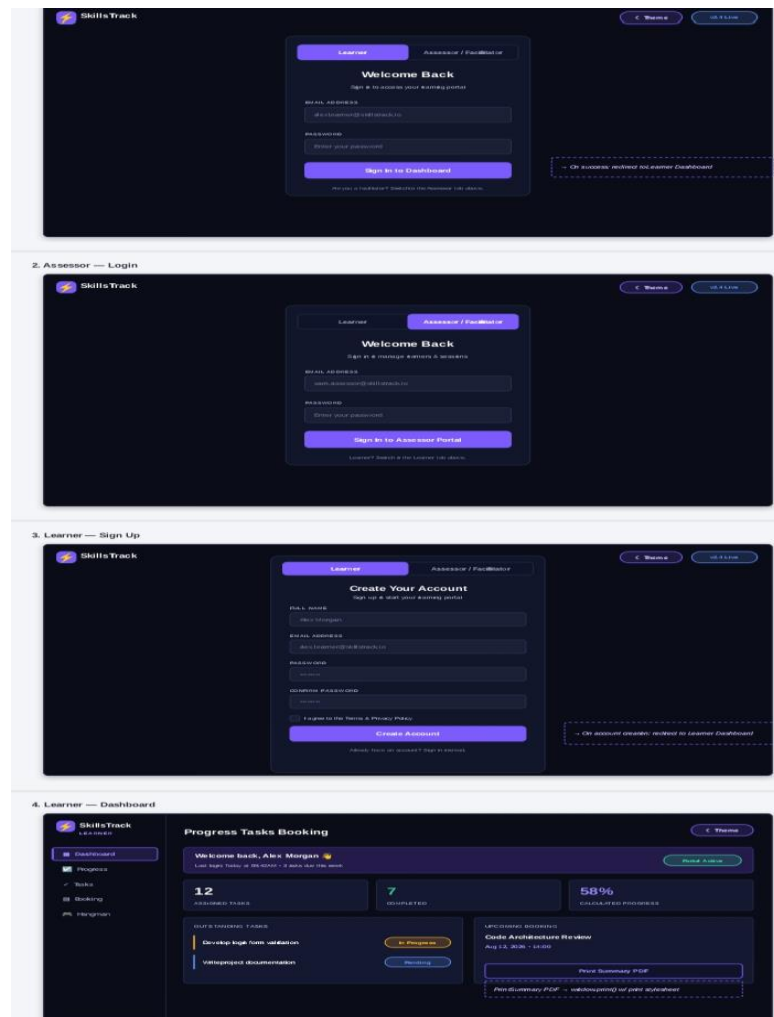
```
document.cookie = "newUser=true"
```

Used to show onboarding tooltip on first dashboard visit.

[P12-4 PRINT]: Print Summary PDF button calls:

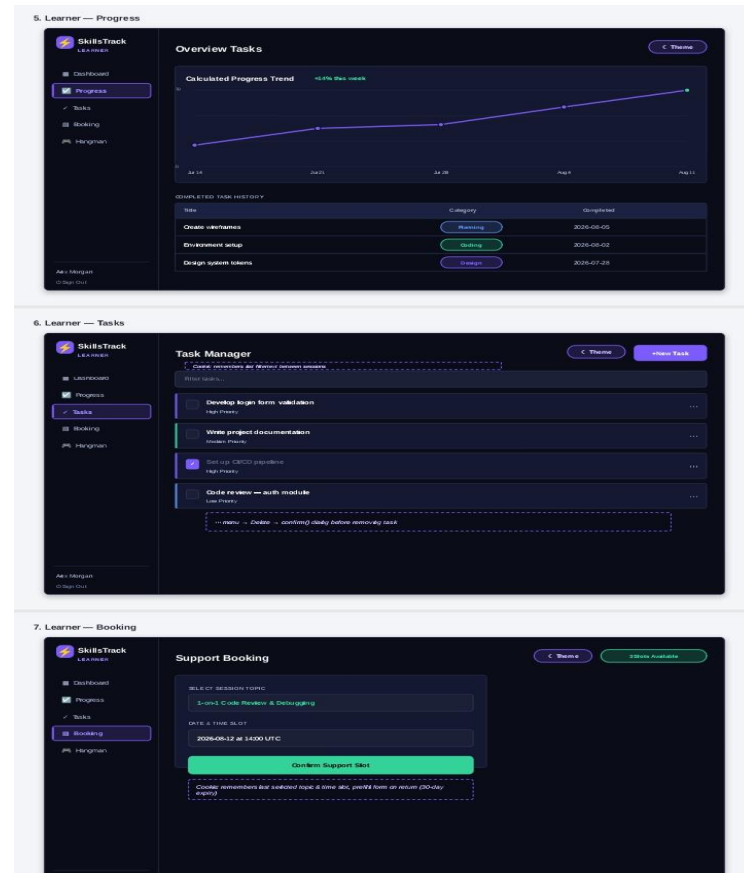
```
window.print() + @media print CSS to hide sidebar/nav and format  
tasks for printing.
```

[P12-1 COOKIE]: Reads cookie to display "Welcome back, Alex Morgan" and last login date.



9

[P12-3 REDIRECT]: On "Confirm Support Slot", show alert then redirect:
`alert("Slot booked!"); window.location.href = "learner-dashboard.html"`

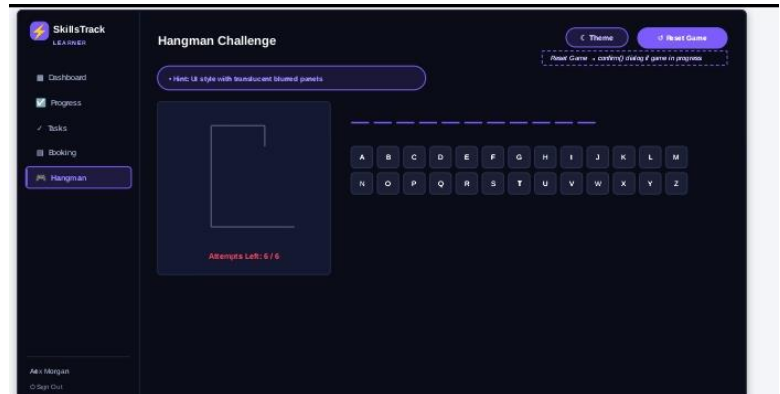


Pseudocode

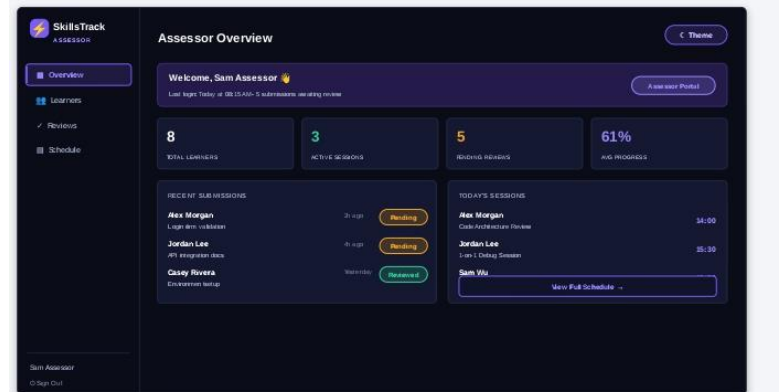
10

[P12-2 DIALOG]: "Reset Game" button triggers:

confirm("Reset current game? Progress will be lost.") before resetting.



9. Assessor — Overview



[P12-1 COOKIE]: Stores assessor preference: "showPendingOnly=true"

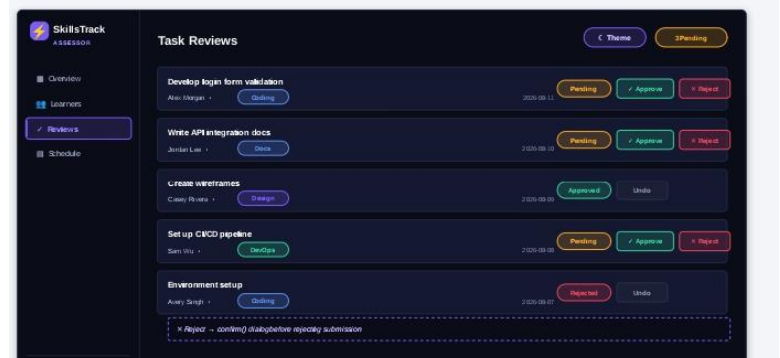
to filter dashboard on next login.

10. Assessor — Learners



[P12-1 COOKIE]: Saves search/filter state in cookie so assessor returns to same view

11. Assessor — Reviews



[P12-2 DIALOG]: "Approve" and "Reject" buttons both trigger:

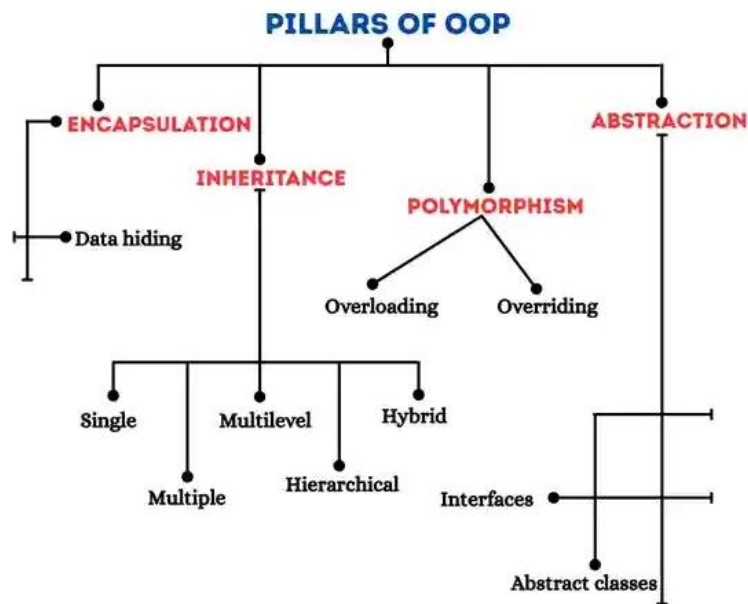
confirm("Are you sure you want to Approve/Reject this submission?")

before submitting review to prevent mistakes.

[P12-3 REDIRECT]: Clicking "View Full Schedule" redirects to detailed calendar view:

window.location.href = "assessor-schedule-full.html"

Pseudocode



OOP STRUCTURE

OOP Structure

We structured the portal using the 4 pillars of OOP shown in our diagram:

1. Encapsulation : User data like 'email', 'password', and 'role' are stored inside a 'User' class with private fields. Access is only through methods like 'login()' and 'getProfile()'. This hides data from direct access.

2. Inheritance : We created a base 'User' class, then extended it into 'Learner' and 'Assessor' classes. 'Learner' inherits from 'User' and adds 'bookSession()'. 'Assessor' inherits and adds 'reviewTask()'.

3. Polymorphism : Both 'Learner' and 'Assessor' override a 'getDashboard()' method from 'User'. The portal calls 'user.getDashboard()' and the correct dashboard loads based on the object type.²

Pseudocode

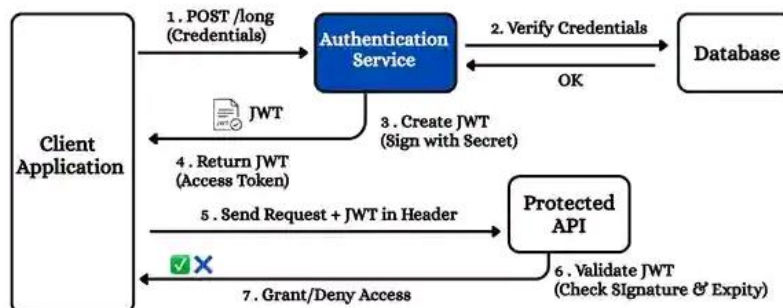
12

Firestore Authentication Flow

Instead of building our own backend, we followed the authentication flow using Firebase Auth:

1. Client POST : The 'Client Application' sends credentials via `signInWithEmailAndPassword()` to Firebase.
2. Verify Credentials: Firebase verifies against its database.
3. Create JWT: On success, Firebase creates and returns an 'ID Token / JWT'.
4. Return Token : The app receives the token and stores it.
5. Send Request : For protected routes like Dashboard, we send requests with the JWT in the Authorization header.
6. Validate : Firebase SDK validates the token signature and expiry.
7. Grant/Deny Access : If valid, the user gets access to protected pages. If not, they are redirected to login.

AUTHENTICATION FLOW



Pseudocode

13

Reflection: GitHub Repository & Collaboration Setup

For P6, our team successfully implemented a professional GitHub collaboration workflow for the SkillsTrack Learner Support Portal. To manage work effectively, we created three separate Project Boards one for Chad, Brian, and Modise each containing assigned Issues in To Do, In Progress, and Done columns. This ensured that all 13 rubric tasks covering coding, design, and documentation were tracked, assigned, and visible to the entire team.

To prevent conflicts and support parallel development, each team member worked on their own feature branch: ``chad``, ``brian``, and ``modise``, with ``main`` kept as the protected default branch. We followed best practice by regularly pulling from ``main``, committing changes locally, and pushing to our remote branches. Evidence shows branches were kept in sync with ``origin`` using ``git status`` and ``git pull``.

All feature work was integrated through Pull Requests. For example, the PR titled "added the figma link into the readme" was reviewed and approved by a teammate before being merged into ``main``. This demonstrates our use of peer code review. We also encountered and resolved a real merge conflict in ``README.md``, using VS Code's merge editor to compare changes and commit the resolved file. This proved our ability to handle collaboration issues.

To maintain code quality, we configured GitHub Actions for Continuous Integration. On every push, a Lint Check workflow runs ESLint automatically to enforce our coding standards. Additionally, we included a ``.gitignore`` file to exclude ``node_modules``, logs, and environment files, and maintained a clear repository folder structure with config, source, and documentation separated.

Overall, P6 shows that our team can use GitHub not just for version control, but for full project management, code review, conflict resolution, and automated quality assurance — skills essential for professional software development.

Pseudocode

14

Pseudocode

15